

1

1) CPU Scheduling Algorithms

```
#include<stdio.h>

struct process
{
    int pid, bt, wt, tat, pr;
};

void fcts()
{
    int n,i;
    struct process p[10];

    printf("Enter number of processes: ");
    scanf("%d",&n);

    for(i=0;<n;i++)
    {
        printf("Enter Burst Time for P%d: ",i+1);
        scanf("%d",&p[i].bt);
        p[i].pid=i+1;
    }

    p[0].wt=0;
    for(i=1;<n;i++)
    {
        p[i].wt=p[i-1].wt+p[i-1].bt;
    }

    printf("\nPID\tBT\tWT\tTAT\n");

    for(i=0;<n;i++)
    {
        p[i].tat=p[i].wt+p[i].bt;
        printf("P%d\t%d\t%d\t%d\n",p[i].pid,p[i].bt,p[i].wt,p[i].tat);
    }
}
```

4

```
void roundRobin()
{
    int n,i,remain,time=0,quantum;
    int bt[10],rt[10],wt[10],tat[10];

    printf("Enter number of processes: ");
    scanf("%d",&n);

    remain=n;

    for(i=0;<n;i++)
    {
        printf("Enter Burst Time for P%d: ",i+1);
        scanf("%d",&bt[i]);
        rt[i]=bt[i];
    }

    printf("Enter Time Quantum: ");
    scanf("%d",&quantum);

    while(remain!=0)
    {
        for(i=0;<n;i++)
        {
            if(rt[i]>0)
            {
                if(rt[i]<quantum)
                {
                    time=time+rt[i];
                    wt[i]=time-bt[i];
                    rt[i]=0;
                    remain--;
                }
                else
                {
                    rt[i]=rt[i]-quantum;
                    time=time+quantum;
                }
            }
        }

        printf("\nPID\tBT\tWT\tTAT\n");

        for(i=0;<n;i++)
        {
            tat[i]=bt[i]+wt[i];
        }
    }
}
```

7

```
scanf("%d",&nu);

for(i=0;<n;i++)
{
    printf("Enter Burst Time for User Process P%d: ",i+1);
    scanf("%d",&user[i].bt);
    user[i].pid=i+1;
}

printf("\n--- System Processes (Higher Priority) ---\n");
calculate(sys,ns);

printf("\n--- User Processes ---\n");
calculate(user,nu);

return 0;
}
```

2

```
void sjf()
{
    int n,i,j;
    struct process p[10],temp;
    printf("Enter number of processes: ");
    scanf("%d",&n);
    for(i=0;<n;i++)
    {
        printf("Enter Burst Time for P%d: ",i+1);
        scanf("%d",&p[i].bt);
        p[i].pid=i+1;
    }
    for(i=0;<n-1;i++)
    {
        for(j=i+1;<n;j++)
        {
            if(p[i].bt > p[j].bt)
            {
                temp=p[i];
                p[i]=p[j];
                p[j]=temp;
            }
        }
    }
    p[0].wt=0;
    for(i=1;<n;i++)
    {
        p[i].wt=p[i-1].wt+p[i-1].bt;
    }
    printf("\nPID\tBT\tWT\tTAT\n");
    for(i=0;<n;i++)
    {
        p[i].tat=p[i].wt+p[i].bt;
        printf("P%d\t%d\t%d\t%d\n",p[i].pid,p[i].bt,p[i].wt,p[i].tat);
    }
}
```

5

```
printf("P%d\t%d\t%d\t%d\n",i+1,bt[i],wt[i],tat[i]);
}

int main()
{
    int ch;

    do
    {
        printf("\nCPU Scheduling Algorithms");
        printf("\n1.FIFS");
        printf("\n2.SJF");
        printf("\n3.Priority");
        printf("\n4.Round Robin");
        printf("\n5.Exit");
        printf("\nEnter choice: ");
        scanf("%d",&ch);

        switch(ch)
        {
            case 1: fcts(); break;
            case 2: sjf(); break;
            case 3: priority(); break;
            case 4: roundRobin(); break;
            case 5: break;
            default: printf("Invalid Choice");
        }

    }while(ch!=5);

    return 0;
}
```

8

3) Real-Time Scheduling (EDF)

```
#include<stdio.h>

struct process
{
    int pid, burst, deadline;
};

int main()
{
    int n,i,j;
    struct process p[10],temp;

    printf("Enter number of processes: ");
    scanf("%d",&n);

    for(i=0;<n;i++)
    {
        printf("Enter Burst Time and Deadline for P%d: ",i+1);
        scanf("%d%d",&p[i].burst,&p[i].deadline);
    }

    p[i].pid=i+1;

    for(i=0;<n-1;i++)
    {
        for(j=i+1;<n;j++)
        {
            if(p[i].deadline > p[j].deadline)
            {
                temp=p[i];
                p[i]=p[j];
                p[j]=temp;
            }
        }
    }

    printf("\nExecution Order using EDF:\n");

    for(i=0;<n;i++)
    {
        printf("P%d ",p[i].pid);
    }

    return 0;
}
```

3

```
void priority()
{
    int n,i,j;
    struct process p[10],temp;

    printf("Enter number of processes: ");
    scanf("%d",&n);

    for(i=0;<n;i++)
    {
        printf("Enter Burst Time and Priority for P%d: ",i+1);
        scanf("%d%d",&p[i].bt,&p[i].pr);
        p[i].pid=i+1;
    }

    for(i=0;<n-1;i++)
    {
        for(j=i+1;<n;j++)
        {
            if(p[i].pr > p[j].pr)
            {
                temp=p[i];
                p[i]=p[j];
                p[j]=temp;
            }
        }
    }

    p[0].wt=0;

    for(i=1;<n;i++)
    {
        p[i].wt=p[i-1].wt+p[i-1].bt;
    }

    printf("\nPID\tPR\tBT\tWT\tTAT\n");

    for(i=0;<n;i++)
    {
        p[i].tat=p[i].wt+p[i].bt;
        printf("P%d\t%d\t%d\t%d\t%d\n",
            p[i].pid,p[i].pr,p[i].bt,p[i].wt,p[i].tat);
    }
}
```

6

2) Multi-Level Queue Scheduling

```
#include<stdio.h>

struct process
{
    int pid, bt, wt, tat;
};

void calculate(struct process p[], int n)
{
    int i;

    p[0].wt=0;
    for(i=1;<n;i++)
    {
        p[i].wt=p[i-1].wt+p[i-1].bt;
    }

    printf("\nPID\tBT\tWT\tTAT\n");

    for(i=0;<n;i++)
    {
        p[i].tat=p[i].wt+p[i].bt;
        printf("P%d\t%d\t%d\t%d\n",
            p[i].pid,p[i].bt,p[i].wt,p[i].tat);
    }
}

int main()
{
    int ns,nu,i;

    struct process sys[10],user[10];

    printf("Enter number of System Processes: ");
    scanf("%d",&ns);

    for(i=0;<ns;i++)
    {
        printf("Enter Burst Time for System Process P%d: ",i+1);
        scanf("%d",&sys[i].bt);
        sys[i].pid=i+1;
    }

    printf("Enter number of User Processes: ");
}
```

9

4) Producer Consumer using Semaphore

```
#include<stdio.h>
struct process
{
    int pid, burst, deadline;
};
int main()
{
    int n,i,j;
    struct process p[10],temp;
    printf("Enter number of processes: ");
    scanf("%d",&n);
    for(i=0;<n;i++)
    {
        printf("Enter Burst Time and Deadline for P%d: ",i+1);
        scanf("%d%d",&p[i].burst,&p[i].deadline);
        p[i].pid=i+1;
    }
    for(i=0;<n-1;i++)
    {
        for(j=i+1;<n;j++)
        {
            if(p[i].deadline > p[j].deadline)
            {
                temp=p[i];
                p[i]=p[j];
                p[j]=temp;
            }
        }
    }

    printf("\nExecution Order using EDF:\n");

    for(i=0;<n;i++)
    {
        printf("P%d ",p[i].pid);
    }

    return 0;
}
```

6) Memory Allocation (First Fit)

```
#include <stdio.h>
int main()
{
    int b[10], p[10], allocation[10];
    int n, np, i;
    printf("Enter number of blocks: ");
    scanf("%d", &n);
    printf("Enter number of processes: ");
    scanf("%d", &np);

    printf("Enter block sizes:\n");
    for(i=0; i<n; i++)
    {
        scanf("%d", &b[i]);
    }

    printf("Enter process sizes:\n");
    for(i=0; i<np; i++)
    {
        scanf("%d", &p[i]);
    }
    for(i=0; i<np; i++)
    {
        allocation[i] = 1;
        for(j=0; j<n; j++)
        {
            if(b[j]>=p[i])
            {
                allocation[j] = 0;
                b[j] = p[i];
                break;
            }
        }
    }

    printf("\nProcess Size\tBlock\n");

    for(i=0; i<np; i++)
    {
        if(allocation[i] == 1)
        {
            printf("P%d\t\tY%d\t\tY%d\n",
                i+1, p[i], allocation[i]+1);
        }
        else
        {
            printf("P%d\t\tY%d\t\tNot Allocated\n",
                i+1, p[i]);
        }
    }

    return 0;
}
```

```
printf("\n");

for(j=0;j<f;j++)
{
    printf("%d\t",frame[j]);
}

printf("\n\nTotal Page Faults = %d",pageFault);

return 0;
}
```